

PORTADA

# **Análisis y Diseño de Sistemas II**

## **Semana 14 — Patrones GoF de Comportamiento I: Strategy, Observer, Command**

Universidad de Antioquía · Ingeniería de Sistemas



Piezas de ajedrez  
representando decisiones y  
comportamientos  
intercambiables

APERTURA

**Ya saben componer objetos sin duplicar código.  
Hoy: cómo hacen que esos objetos *se comporten y se comuniquen sin acoplarse entre sí.***

TRANSICIÓN

## De componer estructuras a coordinar comportamiento

La **semana 13** cerró los 6 patrones **estructurales** de GoF: Adapter, Facade, Composite, Decorator, Proxy y Bridge. Todos resuelven la misma pregunta: *¿cómo se combinan clases y objetos ya existentes en estructuras más grandes, sin duplicar código?*

Hoy se abre la **tercera y última familia** del catálogo GoF: los patrones **de comportamiento**. Ya no se trata de cómo nace un objeto (creacionales) ni de cómo se combina con otros (estructurales), sino de **cómo se distribuyen responsabilidades y algoritmos** entre objetos que colaboran, y de **cómo se comunican** entre sí de forma flexible.

## CONCEPTO

# Qué resuelven los patrones de comportamiento

Los **patrones de comportamiento** son la tercera de las tres familias del catálogo GoF (creacional, estructural, **de comportamiento**). Responden a preguntas como:

- ¿Cómo cambio el **algoritmo** que usa un objeto sin llenar el código de condicionales `if/elif` ?
- ¿Cómo hago que un objeto **avise automáticamente** a otros cuando cambia, sin conocer sus clases concretas?
- ¿Cómo convierto una **acción o solicitud** en algo que puedo guardar, encolar o deshacer, en vez de ejecutarla directo?

El catálogo GoF define **11 patrones de comportamiento** en total. Esta semana se ven **6 de los 11**: hoy, martes, **Strategy, Observer y Command**; el miércoles, **State, Template Method e Iterator**. El viernes se cierra la **Unidad 5** con antipatrones de diseño y un laboratorio final.

TRANSICIÓN

## Primer problema: elegir entre varias formas de hacer lo mismo

Imaginen un sistema de ventas que debe calcular el descuento de una compra, pero existen varias formas de calcularlo: descuento fijo por temporada, descuento porcentual por volumen, o ninguno. La forma más directa —y la más tentadora— es escribir un método con un `if/elif` que decida cuál aplicar según un parámetro.

 Varios caminos posibles  
para llegar al mismo  
resultado

CONTRAEJEMPLO

## El cálculo de descuento resuelto con condicionales

```
class CarritoDeCompras:
    def calcular_descuento(self, tipo, subtotal):
        if tipo == "temporada":
            return 20000
        elif tipo == "volumen":
            return subtotal * 0.15
        elif tipo == "ninguno":
            return 0
        else:
            raise ValueError("tipo desconocido")
```

Cada vez que aparece una nueva forma de calcular el descuento, hay que **modificar** `calcular_descuento()` y agregar otra rama. El método crece indefinidamente, mezcla todos los algoritmos en un solo lugar, y probar uno solo de ellos obliga a leer el método completo.

## CONCEPTO

# Definición: Strategy

**Strategy** (Estrategia) encapsula una **familia de algoritmos intercambiables** detrás de una interfaz común, de modo que el algoritmo usado puede **cambiarse en tiempo de ejecución** sin condicionales que distingan casos.

Cada algoritmo concreto —cada **estrategia**— se implementa en su propia clase, todas cumpliendo la misma interfaz. El objeto que necesita el algoritmo (el **contexto**, en este caso `CarritoDeCompras` ) recibe la estrategia inyectada y simplemente la invoca, sin saber cuál de las implementaciones concretas está usando.

## CONCEPTO

# Strategy, en código

```
interface EstrategiaDescuento:
    def calcular(self, subtotal) -> float: ...

class DescuentoTemporada(EstrategiaDescuento):
    def calcular(self, subtotal):
        return 20000

class DescuentoPorVolumen(EstrategiaDescuento):
    def calcular(self, subtotal):
        return subtotal * 0.15

class SinDescuento(EstrategiaDescuento):
    def calcular(self, subtotal):
        return 0

class CarritoDeCompras:
    def __init__(self, estrategia: EstrategiaDescuento):
        self.estrategia = estrategia    # inyectada

    def calcular_descuento(self, subtotal):
        return self.estrategia.calcular(subtotal)
```

Agregar un descuento nuevo ya no toca `CarritoDeCompras` : se crea otra clase que implementa `EstrategiaDescuento` y se inyecta. El carrito puede incluso **cambiar de estrategia en tiempo de ejecución** asignando otro objeto a `self.estrategia` .

CONCEPTO · RELACIÓN CON LO VISTO

## Strategy y Polymorphism: el patrón GRASP hecho catálogo

En la **semana 10** vieron **Polymorphism** como patrón GRASP: cuando el comportamiento varía según el tipo, la responsabilidad de esa variación se asigna mediante operaciones polimórficas definidas en una interfaz común, en vez de condicionales que pregunten por el tipo.

`EstrategiaDescuento` y sus tres implementaciones son exactamente eso: **polimorfismo** aplicado a la variación de un algoritmo. **Strategy es la forma catalogada y con nombre propio** de aplicar Polymorphism (GRASP) al problema específico de **algoritmos intercambiables** — GRASP da el principio general de asignación de responsabilidades; GoF da la estructura concreta de clases para resolver este caso particular.

TRANSICIÓN

## Segundo problema: avisar a otros sin conocerlos

Strategy resuelve qué algoritmo usar. El segundo problema de hoy es distinto: un objeto cambia de estado (por ejemplo, un pedido pasa de "pendiente" a "confirmado") y varias partes del sistema necesitan enterarse de ese cambio para reaccionar — enviar un correo, actualizar un inventario— sin que el objeto que cambió tenga que **conocer las clases concretas** de quienes reaccionan.

 Una notificación  
avisando a quienes están  
suscritos a un cambio

## CONCEPTO

# Definición: Observer

**Observer** (Observador) define una dependencia **uno-a-muchos** entre objetos: cuando un objeto —el **sujeto** u **observable**— cambia de estado, **notifica automáticamente** a todos los objetos que se registraron como sus **observadores**, sin acoplarse a las clases concretas de esos observadores.

- El sujeto solo conoce una interfaz común de observador (por ejemplo, un método `notificar(evento)` ).
- Cualquier clase puede registrarse como observador implementando esa interfaz, sin que el sujeto sepa nada de ella más allá de eso.
- Los observadores pueden agregarse o quitarse en tiempo de ejecución, sin tocar el sujeto.

## CONCEPTO

# Observer, en código

```
interface ObservadorPedido:
    def notificar(self, pedido, evento): ...

class NotificadorEmail(ObservadorPedido):
    def notificar(self, pedido, evento):
        enviar_correo(pedido.cliente, evento)

class ActualizadorInventario(ObservadorPedido):
    def notificar(self, pedido, evento):
        reservar_stock(pedido.items)

class Pedido:
    def __init__(self):
        self.observadores = []
        self.estado = "pendiente"

    def agregar_observador(self, obs):
        self.observadores.append(obs)

    def confirmar(self):
        self.estado = "confirmado"
        for obs in self.observadores:
            obs.notificar(self, "confirmado")
```

`Pedido` nunca importa `NotificadorEmail` ni `ActualizadorInventario` : solo recorre su lista de `ObservadorPedido` . Se pueden agregar más observadores (un registro de auditoría, por ejemplo) sin modificar `Pedido` .

CONCEPTO · RELACIÓN CON LO VISTO

## Observer y Low Coupling

En la **semana 10** vieron **Low Coupling** como patrón GRASP: minimizar el conocimiento que una clase tiene de otras, para que los cambios en una no se propaguen a las demás. Pedido tendría **alto acoplamiento** si conociera directamente a `NotificadorEmail` y a `ActualizadorInventario`: agregar un tercer observador obligaría a modificarlo.

Observer logra bajo acoplamiento porque el sujeto solo depende de una **interfaz** de observador, nunca de clases concretas. Este mismo principio —notificar cambios sin conocer a quién— es la base conceptual de las **arquitecturas orientadas a eventos** (event-driven), donde sistemas completos se comunican publicando y suscribiéndose a eventos en vez de llamarse directamente. Es solo una mención breve hoy: se profundiza en cursos posteriores.

## INTERACCIÓN

### ¿Strategy u Observer?

En el chat, para cada caso indiquen cuál de los dos patrones calza mejor:

1. Un sistema de envíos que puede calcular el costo de envío por peso, por distancia o por una tarifa plana, y el cliente decide cuál usar al momento de cotizar.
2. Un blog donde, al publicarse un nuevo artículo, deben actualizarse simultáneamente el feed RSS, el índice de búsqueda y la lista de suscriptores por correo.
3. Un videojuego que ofrece tres dificultades con reglas de puntaje distintas, seleccionables por el jugador en cualquier momento.

5 minutos · respondan en el chat

TRANSICIÓN

## Tercer problema: convertir una acción en un objeto

Strategy encapsula un algoritmo. Observer encapsula una notificación. El tercer problema de hoy es distinto: hay operaciones —"agregar un producto", "cancelar un pedido"— que un cliente ejecuta directamente llamando a un método, pero que en ciertos escenarios se necesita **tratar como un objeto**: guardarlas en una cola, ejecutarlas después, deshacerlas, o dejar un registro de qué se hizo.

 Una acción ejecutada  
como si fuera un comando  
de terminal

## CONCEPTO

# Definición: Command

**Command** (Comando) encapsula una **solicitud o acción** como un objeto, permitiendo **parametrizar** a los clientes con distintas solicitudes, **encolarlas**, **deshacerlas** (undo) o **registrarlas** (log) — cosas imposibles si la acción es solo una llamada directa a un método.

Cada acción concreta se convierte en una clase que implementa una interfaz común con un método `ejecutar()`. Un **invocador** (el objeto que dispara la acción, por ejemplo un botón de una interfaz o un controlador) solo conoce esa interfaz común: nunca conoce el detalle interno de cada acción concreta.

## CONCEPTO

# Command, en código

```
interface Comando:
    def ejecutar(self) -> None: ...

class ComandoAgregarProducto(Comando):
    def __init__(self, carrito, producto):
        self.carrito, self.producto = carrito, producto
    def ejecutar(self):
        self.carrito.agregar(self.producto)

class ComandoCancelarPedido(Comando):
    def __init__(self, pedido):
        self.pedido = pedido
    def ejecutar(self):
        self.pedido.cancelar()

class Invocador:
    def __init__(self):
        self.historial = []
    def ejecutar_comando(self, comando: Comando):
        comando.ejecutar()
        self.historial.append(comando)  # queda registrado
```

Invocador llama `ejecutar_comando(comando)` sin importar si el comando agrega un producto o cancela un pedido: solo conoce la interfaz `Comando`. Cada comando ejecutado queda además registrado en `historial`, listo para auditoría o para una futura función de deshacer.

## CONCEPTO

# Command como base de undo y colas de tareas

Que cada acción sea un objeto con su propio estado (los parámetros que recibió) hace posible construir, sobre la misma estructura de Command, dos funcionalidades muy comunes en software real:

- **Deshacer (undo):** si cada comando concreto además implementa un método `deshacer()` con la operación inversa, el invocador puede recorrer su `historial` en reversa y revertir acciones una por una.
- **Colas de tareas:** como un comando es un objeto y no una llamada inmediata, puede guardarse en una lista o en una cola y ejecutarse más tarde, incluso en otro proceso —la base conceptual de sistemas de trabajos en segundo plano (background jobs).

Hoy solo se ve la estructura base de Command (interfaz + comandos concretos + invocador); undo y colas son extensiones naturales del mismo patrón, no patrones distintos.

TENSIÓN CONCEPTUAL

## Strategy sin variación real es sobre-ingeniería

Si `CarritoDeCompras` nunca va a tener más de **una** forma de calcular el descuento —y no hay ninguna variación prevista en el futuro cercano—, crear una interfaz `EstrategiaDescuento` con una sola implementación concreta no resuelve ningún problema real: solo agrega una capa de indirección y una clase más que mantener.

Es la misma tensión que ya vieron con **DIP** (semana 9) y con **Pure Fabrication** (semana 10, GRASP): una abstracción se justifica cuando existe —o es razonablemente previsible— una **variación genuina** que necesita intercambiarse. Strategy se aplica cuando la familia de algoritmos es real, no como plantilla por defecto para "cualquier cálculo".

TENSIÓN CONCEPTUAL

## Observer mal implementado: acoplamiento implícito por eventos

Observer reduce el acoplamiento explícito (el sujeto no importa las clases de sus observadores), pero puede introducir un **acoplamiento implícito** más difícil de rastrear: si un observador, al reaccionar a una notificación, dispara a su vez otra notificación en otro sujeto —que activa otro observador, que activa otro—, se forma una **cadena de notificaciones** cuyo flujo completo ya no es visible leyendo una sola clase.

El código de cada clase individual puede verse limpio y desacoplado; el problema aparece al intentar entender **el comportamiento completo del sistema** en tiempo de ejecución. Por eso Observer exige documentar bien qué reacciona a qué, especialmente cuando hay varias cadenas de notificaciones cruzadas.

 Un patrón de comportamiento en tensión con la sobre-ingeniería o el acoplamiento oculto

## SÍNTESIS

# Los tres primeros patrones de comportamiento, en una frase

1. **De comportamiento:** la tercera y última familia GoF — cómo se distribuyen algoritmos y responsabilidades, y cómo se comunican los objetos que colaboran.
2. **Strategy:** encapsula una familia de algoritmos intercambiables detrás de una interfaz común — la forma catalogada de aplicar Polymorphism (GRASP) a algoritmos que varían.
3. **Observer:** notifica automáticamente a observadores registrados cuando un sujeto cambia de estado, sin conocer sus clases concretas — Low Coupling llevado a la comunicación entre objetos.
4. **Command:** convierte una solicitud en un objeto, permitiendo encolarla, deshacerla o registrarla — base de funcionalidades como undo o colas de tareas.
5. Ninguno se justifica sin un problema real: variación genuina de algoritmo, necesidad real de notificar a varios sin acoplarse, o necesidad real de tratar una acción como dato.

 Síntesis de Strategy,  
Observer y Command

CIERRE

## El miércoles: los otros tres patrones de comportamiento

Hoy vieron 3 de los 6 patrones de comportamiento que cubre esta semana. El **miércoles** se completan los otros tres: **State** (cambiar el comportamiento de un objeto cuando cambia su estado interno, como si cambiara de clase), **Template Method** (definir el esqueleto de un algoritmo dejando que las subclases redefinan ciertos pasos) e **Iterator** (recorrer los elementos de una colección sin exponer su representación interna).

El viernes se cierra la Unidad 5 (Patrones de Diseño y Antipatrones) con antipatrones de diseño y un laboratorio final aplicando el catálogo completo de patrones GoF al proyecto de curso.

 Próxima sesión: los patrones de comportamiento restantes